

课程笔记

CME 295 课程笔记: Lecture 3 Large Language Models

Codex

2026-04-13



视频作者/频道: Stanford Online

发布日期: 2025-10-17

视频时长: 1:48:45

视频链接: <https://www.youtube.com/watch?v=Q5baLehv5So>

目录

1 什么是 LLM：定义、尺度与主干结构	2
1.1 本章小结	2
2 MoE：为什么不必每次都激活全部参数	2
2.1 从“所有参数一起算”到“只选部分专家”	2
2.2 Dense MoE 与 Sparse MoE	3
2.3 MoE 放在 Transformer 的哪里	3
2.4 路由塌缩与负载均衡	3
2.5 本章小结	4
3 从 next-token prediction 到具体解码策略	4
3.1 语言模型真正学到的是什么	4
3.2 Greedy decoding 与 Beam search	5
3.3 Sampling、Top-k 与 Top-p	6
3.4 Temperature 与 guided decoding	6
3.5 本章小结	7
4 上下文窗口与提示策略：从 ICL 到 CoT	8
4.1 Context length 不只是一个数字	8
4.2 In-Context Learning	8
4.3 Chain of Thought 与 Self-Consistency	8
4.4 本章小结	9
5 推理优化：KV cache、PagedAttention 与更快解码	9
5.1 为什么推理会慢	9
5.2 进一步压缩 KV 的存储成本	10
5.3 Speculative decoding 与 Multi-Token Prediction	11
5.4 本章小结	12
6 总结与延伸	13

1 什么是 LLM：定义、尺度与主干结构

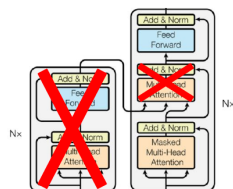
第三讲正式进入大语言模型本体。课程先用上一讲的三类 Transformer-based 模型做 recap，然后把范围收缩到今天最主流的一类：decoder-only、自回归、text-to-text 的大规模语言模型。

讲者对“large”的定义非常工程化，不是模糊形容词，而是至少包含三层含义：

- 参数规模大，通常从十亿级往上走；
- 训练数据量大，常见为数百亿到万亿 token；
- 训练和推理所需算力大，需要大量 GPU 资源和复杂系统优化。

Characteristics

Decoder-only Transformer-based model



Stanford University Figure adapted from "Attention is All You Need", Vaswani et al., 2017.

ICME

图 1: 课程强调现代 LLM 的主干几乎都是 decoder-only Transformer。

课程还特别澄清了一个常见误解：不是所有“大模型”都算今天语境里的 LLM。按照本讲的定义，BERT 这类 encoder-only 模型虽然参数不小，但不做自回归文本生成，因此通常不被归入现代 LLM 的主流范畴。

本讲默认的 LLM 定义

一个现代 LLM 通常是 decoder-only Transformer，通过 next-token prediction 训练，在超大语料和超大算力下学到能够做文本生成的条件分布模型。

1.1 本章小结

LLM 首先是 language model，其次才是 large。课程之所以从结构、数据和算力三个维度同时定义它，是为了说明后面所有技巧都围绕这三个量级约束展开。

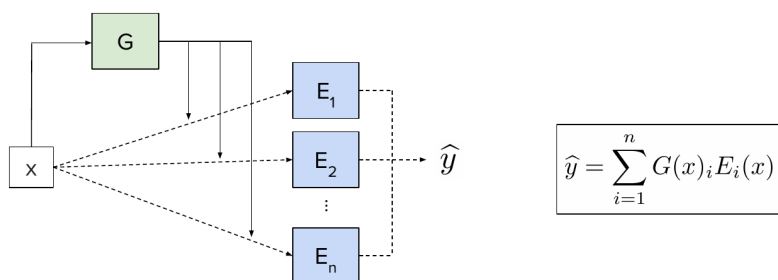
2 MoE：为什么不必每次都激活全部参数

2.1 从“所有参数一起算”到“只选部分专家”

如果一个模型有海量参数，那么每次前向都让所有参数参与计算，会带来极高成本。课程用“房间里有数学家、物理学家、化学家和历史学家，你问一个数学问题是否需要所有人一起回答”这个比喻引出 Mixture of Experts (MoE)。

Overview of MoEs

MoE = Mixture of Experts



Stanford University

ICME

图 2: MoE 的核心结构: router/gate 根据输入选择少量专家参与计算。

在 MoE 中, 给定输入 x , 会有一个 gate 或 router 决定哪些 expert 需要被激活。于是模型的“总参数量”可以很大, 但“单次激活参数量”相对较小, 从而提高参数规模与推理成本之间的比例。

2.2 Dense MoE 与 Sparse MoE

课程区分了两类 MoE:

- Dense MoE: 所有 expert 都参与, 只是权重不同;
- Sparse MoE: 只选极少数 expert, 真正做到稀疏激活。

现代大模型讨论的 MoE, 主要指 sparse MoE。它的价值就在于: 总参数可以继续扩张, 但每个 token 实际只路由到少数专家, 不必让全部参数同时上线。

2.3 MoE 放在 Transformer 的哪里

课程说明, MoE 通常不是替换 attention, 而是替换或增强 FFN 子层。理由很直观: attention 负责 token 间的信息交互, 而 FFN 更像逐位置的特征变换模块, 把 expert 机制放在这里更自然。

2.4 路由塌缩与负载均衡

MoE 的难点不在概念, 而在训练。如果 router 总是把大多数 token 分给同一个 expert, 就会出现 routing collapse, 导致专家利用率极不平衡、学习效率很差。

Training challenges include routing collapse

Symptom. Same expert gets selected most of the time.

"routing collapse"

Stanford University "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity", Fedus et al., 2021.

ICME

图 3: 课程把 routing collapse 定义为大多数 token 总是流向同一专家。

因此训练时通常要加入负载均衡目标, 让不同 expert 获得更均匀的 token 流量。课程在口头说明里也强调, MoE 路由通常是在 token 级别发生的, 即同一序列中的不同 token 完全可能流向不同专家。

Interpreting experts

Layer 0

```
class MoELayer(nn.Module):
    def __init__(self, experts: List[nn.Module]):
        super().__init__()
        assert len(experts) > 0
        self.experts = nn.ModuleList(experts)
        self.gate = gate
        self.args = moe_args

    def forward(self, inputs: torch.Tensor):
        inputs_squashed = inputs.view(-1, inputs.size(-1))
        gate_logits = self.gate(inputs_squashed)
        weights, selected_experts = torch.topk(
            gate_logits, self.args.num_experts_per_token
        )
        weights = nn.functional.softmax(
            weights,
            dim=-1,
            dtype=torch.float,
        ).type_as(inputs)
        results = torch.zeros_like(inputs_squashed)
        for i, expert in enumerate(self.experts):
            batch_idx, nth_expert = torch.where(
                results == weights[batch_idx]
            )
            results_squashed(batch_idx)
        return results.view_as(inputs)
```

Each color represent an expert

Stanford University "Mixture of Experts", Jiang et al., 2024.

ICME

图 4: 按 token 着色的专家路由图说明, 不同 token 会被送往不同 expert。

2.5 本章小结

MoE 的核心不是“多几个模块”, 而是把模型扩张方式从“所有参数永远一起工作”改成“有条件地激活部分参数”。它让参数规模继续上升成为可能, 但代价是必须解决路由训练、负载均衡和系统实现问题。

3 从 next-token prediction 到具体解码策略

3.1 语言模型真正学到的是什么

课程从最基本的 next-token prediction 出发, 强调语言模型学习的是条件分布:

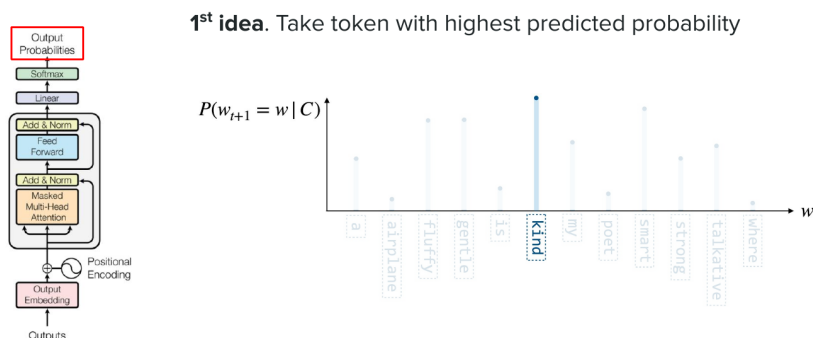
$$p_{\theta}(x_t | x_{<t})$$

其中:

- $x_{<t}$: 当前位置之前的 token 前缀;
- x_t : 当前位置候选 token;
- p_θ : 模型参数为 θ 的条件概率分布。

训练时模型学的是这个分布; 推理时真正困难的, 是如何从这个分布中选 token。课程用 greedy、beam search 与 sampling 三种策略逐步展开。

Predicting next token with greedy decoding



Stanford University

Figure adapted from "Attention is All You Need", Vaswani et al., 2017. "Super Study Guide: Transformers and Large Language Models", Amidi et al., 2024.

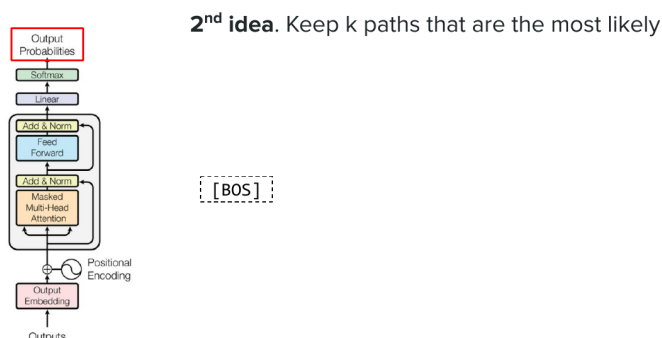
ICME

图 5: Greedy decoding 每一步都选当前概率最高的 token。

3.2 Greedy decoding 与 Beam search

Greedy decoding 最简单, 但只追求局部最优, 容易在整体序列层面不自然或不够多样。Beam search 则保留多个候选路径, 尝试避免局部最优陷阱。

Predicting next token with beam search



Stanford University

Figure adapted from "Attention is All You Need", Vaswani et al., 2017.

ICME

图 6: Beam search 通过维护多个候选前缀, 寻找更高整体概率的序列。

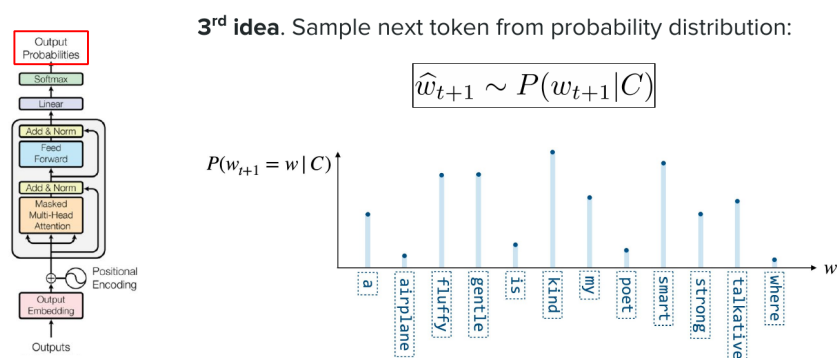
课程也提醒 beam search 并不是今天开放式生成的默认选择，因为它更适合机器翻译等追求高条件概率序列的任务，而开放式对话、创作往往更需要多样性和自然性。

3.3 Sampling、Top-k 与 Top-p

为了保留多样性，生成系统常用 sampling：不是永远取最高概率 token，而是按分布采样。进一步地：

- Top-k：只保留前 k 个最可能 token 后再采样；
- Top-p：保留累计概率达到 p 的最小 token 集合后再采样。

Predicting next token with sampling



Stanford University

"Super Study Guide: Transformers and Large Language Models", Amidi et al., 2024.

ICME

图 7: Sampling 不再固定选择最大概率 token，而是在概率分布上抽样。

这两种方法都在做同一件事：把极低概率、往往会导致胡言乱语的长尾 token 剪掉，同时保留一定随机性。

3.4 Temperature 与 guided decoding

temperature 用来调节分布尖锐程度：

$$p_i^{(T)} = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

符号说明如下：

- z_i ：第 i 个 token 的 logit；
- T ：温度；
- $T < 1$ 时分布更尖锐，更保守；
- $T > 1$ 时分布更平坦，更发散。

Impact of temperature on probabilities

Small T

?

High T

?

Stanford University

ICME

图 8: 温度并不改变 token 排序本身, 而是改变概率分布的尖锐程度。

guided decoding 则是另一条路线: 不是调分布, 而是硬性约束哪些 token 合法。课程用生成 JSON 的例子说明, 如果输出必须满足某种格式, 那么可以在每一步只允许“语法上合法”的 token 进入候选集合。

Constraining the output via guided decoding

Motivation. Generate output in a specific format

Input prompt

*Generate a description of my
33-year old teddy bear who
likes reading.*

Do this in JSON format.

Desired output (JSON)

```
{  
  "first_name": "teddy",  
  "last_name": "bear",  
  "age": 33,  
  "hobby": "reading"  
}
```

Stanford University

ICME

图 9: Guided decoding 通过约束合法 token 集, 强制输出满足预定结构。

3.5 本章小结

模型训练学的是条件分布, 推理系统解决的是“如何从分布中取样”。Greedy、beam、sampling、temperature 和 guided decoding 都在处理这一层。生成质量不仅取决于模型本身, 也取决于解码策略。

4 上下文窗口与提示策略：从 ICL 到 CoT

4.1 Context length 不只是一个数字

课程把 context length 定义为模型在单次前向中能看到的输入长度。它决定了提示词、检索文档、历史对话和中间推理痕迹能否同时放进上下文窗口。随着应用复杂化，context length 直接限制了模型能做多长链条的条件推理。

4.2 In-Context Learning

ICL (In-Context Learning) 指的是不改模型参数，只通过 prompt 中给出的指令和示例，让模型在当前上下文里“表现得像学会了任务”。



Stanford University

Language Models are Few-Shot Learners, Brown et al., 2020.

ICME

图 10: ICL 通过在 prompt 里放入任务说明与示例，让模型在上下文中完成适配。

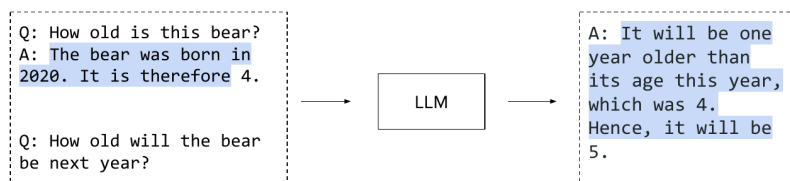
课程把 zero-shot、one-shot、few-shot 都放在这一框架里理解：区别只在 prompt 里是否给例子、给几个例子，而不是模型内部结构发生变化。

4.3 Chain of Thought 与 Self-Consistency

Chain of Thought (CoT) 的核心观察是：对于复杂推理任务，让模型显式生成中间推理步骤，往往比直接输出最终答案更稳。

Chain of thought

Idea. Explaining reasoning helps in improving performance.



Discussion.

- Interpretability + explanation
- More tokens: higher cost and latency

Stanford University

"Chain-of-Thought Prompting Elicits Reasoning in Large Language Models", Wei et al., 2022.

ICME

图 11: CoT 鼓励模型先写出中间推理过程，再给出最终答案。

但 CoT 也会带来更长输出和更高成本，所以课程进一步引出 self-consistency：与其只走一条推理链，不如采样多条 reasoning path，再对最终答案做聚合。这样做的逻辑是，单条链可能偶然走偏，但多个独立样本的共同结论往往更稳。

4.4 本章小结

Prompting 技术的共同点是：不改参数、改输入。ICL 解决快速适配，CoT 解决复杂推理可分解性，self-consistency 则用多路径聚合降低单次采样的偶然性。这些策略解释了为什么“会写 prompt”在 LLM 时代成为真实能力。

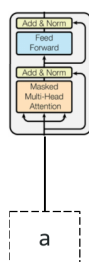
5 推理优化：KV cache、PagedAttention 与更快解码

5.1 为什么推理会慢

课程明确指出，推理阶段的瓶颈往往是 memory-bound，而不是 compute-bound。生成新 token 时，模型需要反复访问之前所有 token 的 key/value；如果每次都从头计算，代价会很高。

Reusing computations with KV caching

Motivation. New token needs to interact with all previous tokens.



Stanford University Figure adapted from "Attention is All You Need", Vaswani et al., 2017.

ICME

图 12: KV caching 通过缓存历史 key/value, 避免每生成一个 token 都重算全部过去。

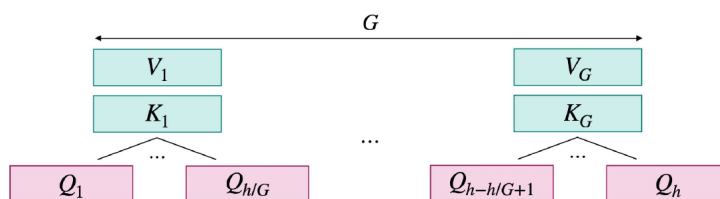
KV cache 的基本思路就是把历史 token 的 key/value 存下来。这样生成第 t 个 token 时, 只需计算最新 token 的投影, 并与缓存交互。

5.2 进一步压缩 KV 的存储成本

缓存历史当然会节省重算, 但也会消耗大量显存。于是课程把上一讲的 MQA/GQA 再拉回来, 说明共享 key/value 头不仅是注意力近似技巧, 也是 KV cache 压缩手段。

Sharing attention heads

Idea. Share key/value attention heads within groups of queries



Stanford University Figure from "Super Study Guide: Transformers & Large Language Models", Amidi et al., 2024.

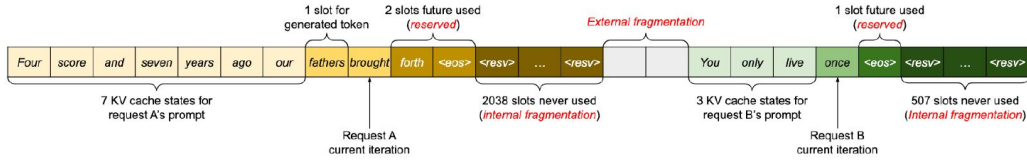
ICME

图 13: MQA/GQA 减少了需要缓存的 key/value 副本数, 从而降低推理内存。

进一步地, PagedAttention 针对服务系统层的内存碎片问题, 把 KV 以分页方式管理, 减少浪费并提高吞吐。

Manage memory with PagedAttention

Observation. Lots of memory waste when storing KV cache.



Stanford University Figure from "Efficient Memory Management for Large Language Model Serving with PagedAttention", by Kwon et al., 2023.

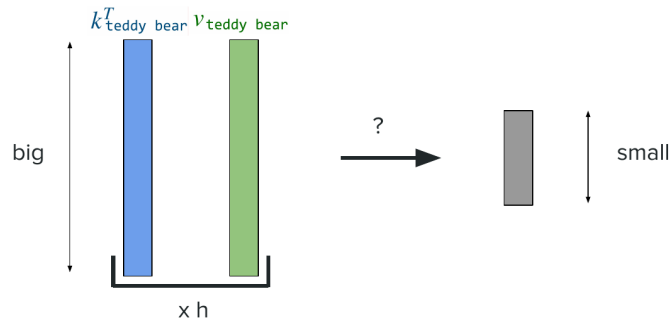
ICME

图 14: PagedAttention 把 KV cache 做成分页管理，降低服务系统中的显存浪费。

课程还介绍了 latent attention 思路：不是缓存完整 K/V，而是缓存更低维的压缩表示，换取更小内存占用。

Reduce memory of KV cache with latent attention

Solution. Store compressed representations instead!



Stanford University "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model", by DeepSeek, 2023.

ICME

图 15: Latent attention 通过缓存压缩后的表示，进一步减小 KV cache 的内存占用。

5.3 Speculative decoding 与 Multi-Token Prediction

另一条加速路线不是压 KV，而是减少大模型逐 token 串行生成的等待时间。Speculative decoding 先让一个小 draft model 连续猜若干 token，再由大模型一次性验证。如果都被接受，就相当于一次前向推进多个 token。

Speeding up decoding with speculative decoding

Idea. Use a draft (small) model to generate tokens validated by a target (big) model.

Stanford University "Accelerating Large Language Model Decoding with Speculative Sampling", Chen et al., 2023.

ICME

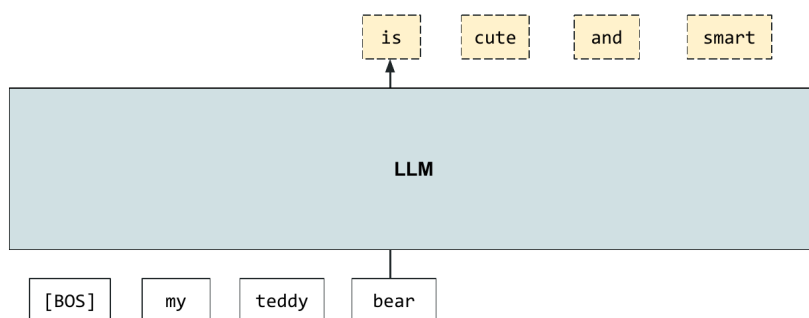
图 16: Speculative decoding 用小模型起草 token，再由大模型并行验证。

课程特别强调它背后的概率论保证：通过接受-拒绝机制，可以让最终采样结果仍然匹配目标大模型分布，而不是仅仅得到一个启发式近似。

Multi-Token Prediction (MTP) 则把“草稿模型”进一步吸收到同一模型内部，通过多个预测头一次预测多个未来 token，再由主头校验。它改变的是训练目标，因此和 speculative decoding 在原理上相近，但不完全等价。

Generate several tokens at once via MTP

MTP = Multi-Token Prediction



Stanford University "Better & Faster Large Language Models via Multi-token Prediction", Gloeckle et al., 2024.

ICME

图 17: MTP 在同一模型中加入多个预测头，以训练时的多 token 目标换取更快生成。

5.4 本章小结

推理优化的主线可以概括为两类：一类是少算或不重算，例如 KV cache、MQA/GQA、latent attention；另一类是更快推进生成步数，例如 speculative decoding 与 MTP。两类方法共同服务于同一个目标：在不明显损伤质量的前提下，提高吞吐、降低延迟。

6 总结与延伸

第三讲把“Transformer 课程”真正推进到了“LLM 课程”。回顾整讲，最值得沉淀的是五条主线：

1. **定义主线**：现代 LLM 通常是大规模 decoder-only 自回归模型。
2. **扩展主线**：MoE 通过稀疏激活扩大总参数规模，而不让单次激活成本同步爆炸。
3. **生成主线**：模型输出的是条件分布，真正的文本风格与稳定性很大程度上由解码策略决定。
4. **提示主线**：ICL、CoT、self-consistency 表明，不改参数也能显著改变任务表现。
5. **系统主线**：当模型足够大时，推理瓶颈进入内存与服务系统层，KV cache、PagedAttention、speculative decoding 等技巧因此成为核心基础设施。

更重要的是，这些内容并不是分散的技巧库。它们一起刻画出今天 LLM 的真实形态：一方面，模型在结构上仍然站在 Transformer 的主干之上；另一方面，真正让它“可用”的，是大量围绕路由、采样、上下文和系统吞吐做出的工程决策。后续继续学习训练、对齐、agent 和评测时，这一讲给出的生成与推理框架会不断反复出现。